

Retail Sales Data Wrangling & Summarization

Cameron Batts | pandas · DataFrame Methods · Merging · GroupBy · Method Chaining

```
In [3]: import pandas as pd  
import numpy as np
```

```
In [4]: sales = pd.read_csv("sales.csv")  
  
stores = pd.read_csv("stores.csv")
```

```
In [5]: stores.head()
```

```
Out[5]:
```

	Store	Type	Size
0	1	A	151315
1	2	A	202307
2	3	B	37392
3	4	A	205863
4	5	B	34875

```
In [6]: stores.tail()
```

```
Out[6]:
```

	Store	Type	Size
40	41	A	196321
41	42	C	39690
42	43	C	41062
43	44	C	39910
44	45	B	118221

```
In [7]: stores.info
```

```
Out[7]: <bound method DataFrame.info of
```

			Store Type	Size
0	1	A	151315	
1	2	A	202307	
2	3	B	37392	
3	4	A	205863	
4	5	B	34875	
5	6	A	202505	
6	7	B	70713	
7	8	A	155078	
8	9	B	125833	
9	10	B	126512	
10	11	A	207499	
11	12	B	112238	
12	13	A	219622	
13	14	A	200898	
14	15	B	123737	
15	16	B	57197	
16	17	B	93188	
17	18	B	120653	
18	19	A	203819	
19	20	A	203742	
20	21	B	140167	
21	22	B	119557	
22	23	B	114533	
23	24	A	203819	
24	25	B	128107	
25	26	A	152513	
26	27	A	204184	
27	28	A	206302	
28	29	B	93638	
29	30	C	42988	
30	31	A	203750	
31	32	A	203007	
32	33	A	39690	
33	34	A	158114	
34	35	B	103681	
35	36	A	39910	
36	37	C	39910	
37	38	C	39690	
38	39	A	184109	
39	40	A	155083	
40	41	A	196321	
41	42	C	39690	
42	43	C	41062	
43	44	C	39910	
44	45	B	118221>	

```
In [8]: stores.value_counts
```

```
Out[8]: <bound method DataFrame.value_counts of
0      1      A  151315
1      2      A  202307
2      3      B   37392
3      4      A  205863
4      5      B   34875
5      6      A  202505
6      7      B   70713
7      8      A  155078
8      9      B  125833
9     10      B  126512
10     11      A  207499
11     12      B  112238
12     13      A  219622
13     14      A  200898
14     15      B  123737
15     16      B   57197
16     17      B   93188
17     18      B  120653
18     19      A  203819
19     20      A  203742
20     21      B  140167
21     22      B  119557
22     23      B  114533
23     24      A  203819
24     25      B  128107
25     26      A  152513
26     27      A  204184
27     28      A  206302
28     29      B   93638
29     30      C   42988
30     31      A  203750
31     32      A  203007
32     33      A   39690
33     34      A  158114
34     35      B  103681
35     36      A   39910
36     37      C   39910
37     38      C   39690
38     39      A  184109
39     40      A  155083
40     41      A  196321
41     42      C   39690
42     43      C   41062
43     44      C   39910
44     45      B  118221>
```

We see there are different type of store from A,B, & C. Additionally we see that each store varies in the size of the store.

```
In [9]: sales['IsHoliday'] = sales['IsHoliday'].astype(int)
```

```
In [10]: sales.query("IsHoliday == 0")
```

```
Out[10]:
```

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	05/02/2010	24924.50	0
2	1	1	19/02/2010	41595.55	0
3	1	1	26/02/2010	19403.54	0
4	1	1	05/03/2010	21827.90	0
5	1	1	12/03/2010	21043.39	0
...	...	...	...	...	...
421565	45	98	28/09/2012	508.37	0
421566	45	98	05/10/2012	628.10	0
421567	45	98	12/10/2012	1061.02	0
421568	45	98	19/10/2012	760.01	0
421569	45	98	26/10/2012	1076.80	0

391909 rows × 5 columns

```
In [11]: sales_2b = sales.loc[sales['IsHoliday'] == 0]

sales_2b.head()
```

```
Out[11]:
```

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	05/02/2010	24924.50	0
2	1	1	19/02/2010	41595.55	0
3	1	1	26/02/2010	19403.54	0
4	1	1	05/03/2010	21827.90	0
5	1	1	12/03/2010	21043.39	0

Merge stores and sales Used a left-join which attaches store information to each sales row

I used a left-join which keeps all rows from the sales_2b dataset and brings the matching information to the store data set. Inner-Join would only keep rows with matching store values in both data frames which would cause some sales to drop. Right-Join would keep all stores even those with no sales. Outer-Join would keep every row from both tables which would create many missing values

```
In [12]: # Merge stores and sales

# Used a left-join which attaches store information to each sales row
sales_stores = sales_2b.merge(stores, on='Store', how='left')

sales_stores.head()
```

```
Out[12]:
```

	Store	Dept	Date	Weekly_Sales	IsHoliday	Type	Size
0	1	1	05/02/2010	24924.50	0	A	151315
1	1	1	19/02/2010	41595.55	0	A	151315
2	1	1	26/02/2010	19403.54	0	A	151315
3	1	1	05/03/2010	21827.90	0	A	151315
4	1	1	12/03/2010	21043.39	0	A	151315

```
In [13]: # Create a new dataframe that summarizes weekly sales by Store, Date,
weekly_store_sales = (sales_stores
    # Group the data by Store, Date, and store Type
    .groupby(['Store', 'Date', 'Type'])['Weekly_Sale
    # For each group, sum the Weekly_Sales values
    .sum()
    # Turn the grouped index back into regular column
    .reset_index()
)
```

```
In [14]: weekly_store_sales['Weekly_Sales'].mean()
```

```
Out[14]: np.float64(1041256.3802088555)
```

Observations

We can see that Type A has the highest avg sales with around 1.37M followed by Type B with 816K and lastly Type C with 472K which indicates that store type clearly has a big influence on sales.

```
In [15]: Chained_Weekly_Sales = (
    sales
    .query("IsHoliday == 0")
    .merge(stores, on='Store', how='left')
    .groupby(['Store', 'Date', 'Type'])['Weekly_Sales']
    .sum()
    .reset_index()
)

Chained_Weekly_Sales.head()
```

Out[15]:

	Store	Date	Type	Weekly_Sales
0	1	01/04/2011	A	1495064.75
1	1	01/06/2012	A	1624477.58
2	1	01/07/2011	A	1488538.09
3	1	01/10/2010	A	1453329.50
4	1	02/03/2012	A	1688420.76